1) Division and Array Access

Write a Java program that:

- Takes two integers from the user.
- Divides the first number by the second number.
- Stores the result in an array of size 3 at an index given by the user.

Handle possible exceptions using **multiple catch blocks**:

- division by zero
- invalid array index
- invalid input type

2) String to Number + Division

Write a program that:

- Reads two values as strings.
- Converts them to integers.
- Divides the first by the second.

Use **multiple catch** to handle:

- invalid number format
- division by zero
- any other unexpected exception

3) File Name Length and Character Access

Write a program that:

- Takes a string from the user.
- Prints the character at a user-given index.
- Converts the string into an integer and prints double its value.

Use **multiple catch** blocks for:

- null string (if input is treated as null in your logic)

- string index out of range
- number format issue

4) Student Marks Calculator

Write a program that:

- Creates an integer array of marks (size 5).
- Takes an index from the user and prints the mark.
- Then divides that mark by another user-provided number.

Use **multiple catch** to handle:

- invalid index
- division by zero
- input mismatch

5) Simple Banking Withdrawal

Write a program that:

- Stores account balance as an integer.
- Takes withdrawal amount and a divisor value.
- Calculates remainingBalance / divisor and prints the result.

Use **multiple catch** blocks for:

- arithmetic exception
- illegal argument (if withdrawal amount is negative, throw it manually)
- generic exception

6) Product Price Parser

Write a program that:

- Reads price as string (example: "250").
- Converts it to integer.
- Stores it in an array at a given position.
- Prints 1000 / price.

Use **multiple catch** blocks to handle:

- number format exception
- array index out of bounds
- arithmetic exception

8) Nested Try: String Parsing and Character Access

Write a program that:

- Takes a string input.
- In one inner try, gets a character from a user-given position.
- In another inner try, converts the same string to an integer.

Use nested try-catch to separately handle:

- string index errors
- number format errors
- outer-level unexpected exceptions

9) Nested Try in Loop

Write a program that:

- Runs a loop 3 times.
- Each time, asks the user for an array index and a divisor.
- In nested try blocks:
    - access an array value
    - divide by the divisor

Requirements:

- If one iteration fails, the loop should continue.
- Handle array and arithmetic exceptions separately using nested try-catch.

10) Nested Try with Manual Exception

Write a program that:

- Takes age and a number from the user.
- In one inner try, check age:
    - if age < 18, throw an exception manually
- In another inner try, divide 100 by the entered number

Use nested try-catch to handle:

- underage/custom exception
- arithmetic exception
- outer catch for input mismatch or other issues

11) File Simulation (Without Real File Handling)

Write a program that simulates file processing using strings:

- First input: “line number” (array index in a string array)
- Second input: “value” inside that line (convert to integer)
- Third input: divisor

Use nested try-catch:

- inner try 1: access line from array
- inner try 2: parse integer from selected line
- inner try 3: divide parsed number by divisor

Handle each error in its own catch.

12) Menu-Based Nested Try Program

Write a menu program with options:

1. Divide two numbers
2. Access array element
3. Convert string to integer

For each option:

- Use an outer try for menu selection and flow

- Use inner try-catch for the selected operation

Requirements:

- Invalid menu choice should be handled
- Each operation should have its own exception handling
- Program should not crash on one wrong input

13) Employee Salary Processor

Write a program that:

- Reads salary as string
- Converts to integer
- Stores in an array by index
- Divides annual salary by user-given months

Requirements:

- Use **nested try-catch** for parsing, storing, and division
- Use **multiple catch** in at least one try block
- Print meaningful error messages